

哈尔滨工业大学 法国里昂商学院
Harbin Institute of Technology emlyon business school

Python BootCamp

Complete Command Reference

全课程命令速查表

黄子扬 Ziyang Huang

二叉苹果树

Course Code: 22EM31202C • Object Oriented Programming

June 13, 2026

Contents

1	Basic Syntax & Operators / 基础语法与运算符	2
2	Strings / 字符串	4
3	Control Structures / 控制结构	6
4	Data Structures / 数据结构	7
5	Modules & Packages / 模块与包	10
6	NumPy / 数值计算	12
7	Object-Oriented Programming / 面向对象编程	14
8	File I/O & Error Handling / 文件 IO 与错误处理	16
9	PEP-8 Style Guide / PEP-8 编码规范	18
10	Matplotlib / 数据可视化	19
11	Pandas / 数据分析	22
12	Seaborn / 统计可视化	27
13	Practice Set Patterns / 练习题常用模式	28
14	Python Built-in Functions / Python 内置函数速查	29

1 Basic Syntax & Operators / 基础语法与运算符

1.1 Variables & Assignment / 变量与赋值

Syntax	Description / 说明
<code>x = 10</code>	Assignment / 赋值: 右侧值绑定到左侧变量名
<code>x, y = 1, 2</code>	Multiple assignment / 多重赋值 (同时绑定多个变量)
<code>a = b = 0</code>	Chain assignment / 链式赋值 (a 和 b 都指向 0)
<code>del x</code>	Delete variable / 删除变量, 释放引用
<code>None</code>	Null value (NoneType) / 空值——不等于 0、False 或 ""
<code>type(x)</code>	Return data type / 返回变量的数据类型
<code>isinstance(x, int)</code>	Check if x is of given type / 类型检查 (推荐, 支持继承判断)
<code>id(x)</code>	Memory address / 返回对象的内存地址 (整数)

1.2 Data Types / 数据类型

Syntax	Description / 说明
<code>int(x)</code>	Convert to integer / 转整数; <code>int(9.9) → 9</code> (截断, 非四舍五入)
<code>float(x)</code>	Convert to float / 转浮点数
<code>str(x)</code>	Convert to string / 转字符串
<code>bool(x)</code>	Convert to boolean / 转布尔值
<code>bool(0) → False</code>	Falsy values / 假值: <code>0</code> , <code>0.0</code> , <code>""</code> , <code>None</code> , <code>[]</code> , <code>{}</code> , <code>False</code>
<code>int("101", 2) → 5</code>	Convert from base / 从指定进制转换 (2= 二进制, 16= 十六进制)

1.3 Arithmetic & Comparison Operators / 算术与比较运算符

Operator	Description / 说明
<code>+</code> <code>-</code> <code>*</code>	Addition / Subtraction / Multiplication (加/减/乘)
<code>/</code>	Standard division $\rightarrow$ float / 标准除法 (总是返回 float)
<code>//</code>	Floor division / 整除 (向下取整, 丢弃余数)
<code>**</code>	Exponentiation / 幂运算: $3**2 \rightarrow 9$
<code>%</code>	Modulo (remainder) / 取模 (余数): $7\%3 \rightarrow 1$
<code>==</code> <code>!=</code>	Equal / Not equal (等于/不等于)
<code><</code> <code>></code> <code><=</code> <code>>=</code>	Comparison operators / 比较运算符 (返回 bool)
<code>and</code> <code>or</code> <code>not</code>	Logical operators / 逻辑运算符
<code>&</code> <code> </code> <code>~</code>	Bitwise AND/OR/NOT / 位运算——Pandas 中 <code>&</code> / <code> </code> 用于逐元素逻辑
<code>is</code> <code>is not</code>	Identity comparison / 身份比较 (比较内存地址, 非值)
<code>in</code> <code>not in</code>	Membership test / 成员检查

1.4 print() & input() / 输入与输出

Syntax	Description / 说明
<code>print(x)</code>	Print x to console / 打印到控制台
<code>print(x, y, sep=',')</code>	Custom separator / 自定义分隔符 (默认空格)
<code>print(x, end='!')</code>	Custom end char / 自定义结尾符 (默认 <code>\n</code> 换行)
<code>input("Prompt: ")</code>	Read user input / 读取用户输入——总是返回 <code>str</code>
<code># comment</code>	Single-line comment / 单行注释

2 Strings / 字符串

2.1 String Basics / 字符串基础

Syntax	Description / 说明
"hello" / 'hello'	Single/double quotes / 单引号或双引号包裹
len(s)	String length (includes spaces) / 字符串长度 (含空格)
s[i]	Positive index / 正索引 (从 0 开始)
s[-i]	Negative index / 负索引 (-1 = 最后一个字符)
s[start:stop]	Slice / 切片——左闭右开 (含 start, 不含 stop)
s[start:stop:step]	Slice with step / 带步长的切片
s[::-1]	Reverse string / 反转字符串

2.2 String Methods / 字符串方法

Method	Description / 说明
s.lower() / s.upper()	Convert case / 转小写/大写
s.title() / s.capitalize()	Title case / Capitalize first letter / 首字母大写
s.strip()	Remove leading+trailing whitespace / 去除首尾空白
s.lstrip() / s.rstrip()	Remove left/right whitespace only
s.replace(old, new)	Replace all occurrences / 替换所有匹配——返回新字符串
s.split(delim)	Split by delimiter → list / 按分隔符拆分为列表
delim.join(list)	Join list elements with delimiter / 用分隔符连接列表
s.count(sub)	Count occurrences of sub / 统计子串出现次数
s.find(sub)	Find first index of sub / 查找子串——找不到返回 -1 (不报错)
s.index(sub)	Find first index—raises ValueError if not found / 找不到则报错
s.startswith(p)	Check prefix / 检查是否以 p 开头 → bool
s.endswith(p)	Check suffix / 检查是否以 p 结尾 → bool
s.isdigit() / s.isalpha()	Check all digits / all letters / 全数字/全字母检查
s.isnumeric() / s.isalnum()	Check numeric / alphanumeric / 数值/字母数字检查
s.zfill(width)	Pad with leading zeros / 左侧补零至指定宽度

Immutability / 不可变性: Strings are immutable —all methods return **new** strings.

字符串不可变——所有方法返回新字符串，必须重新赋值才能保留结果。

2.3 String Formatting / 字符串格式化

Syntax	Description / 说明
<code>f"{var}"</code>	f-string (Best Practice / 推荐) ——Python 3.6+, 自动转换类型
<code>f"{var:.2f}"</code>	Format float to 2 decimal places / 保留两位小数
<code>f"{var:>10}"</code>	Right-align in 10-char width / 右对齐, 宽度 10
<code>"{} {}".format(a, b)</code>	<code>.format()</code> method / 经典格式化方法
<code>"%s %d %f" % (s, d, f)</code>	Old-style <code>%</code> formatting / 旧式格式化 (不推荐)
<code>r"raw\string"</code>	Raw string / 原始字符串——反斜杠不转义

3 Control Structures / 控制结构

3.1 Conditionals / 条件判断

Syntax	Description / 说明
<code>if cond: ...</code>	Basic if / 基本条件——条件为 True 时执行缩进块
<code>if cond: ... else: ...</code>	If-else / 二选一分支
<code>if c1: ... elif c2: ... else: ...</code>	Full chain / 完整链——从上到下检查， 首个为真即执行
<code>x if cond else y</code>	Ternary operator / 三元表达式——简单二选一赋值
<code>pass</code>	Placeholder / 占位符——什么都不做，避免语法错误

3.2 Loops / 循环

Syntax	Description / 说明
<code>for item in iterable: ...</code>	Iterate over iterable / 遍历可迭代对象中的每个元素
<code>for i in range(n): ...</code>	Loop n times (0 to n-1) / 循环 n 次
<code>range(start, stop, step)</code>	Integer sequence / 整数序列—— 左闭右开
<code>for i, v in enumerate(lst):</code>	Loop with index + value / 同时获取索引和值
<code>for a, b in zip(l1, l2):</code>	Parallel iteration / 并行遍历多个序列
<code>while cond: ...</code>	Repeat while cond is True / 条件循环—— 小心无限循环
<code>break</code>	Exit loop immediately / 立即退出整个循环
<code>continue</code>	Skip to next iteration / 跳过本次迭代剩余部分
<code>else (with for/while)</code>	Runs if loop completes without break / 循环正常完成时执行

3.3 Functions / 函数

Syntax	Description / 说明
<code>def f(a, b): return x</code>	Function definition / 函数定义
<code>def f(a, b=default): ...</code>	Default argument / 带默认值的参数
<code>def f(*args, **kwargs): ...</code>	Variable arguments / 可变参数—— <code>*args</code> 打包为元组， <code>**kwargs</code> 打包为字典
<code>return x</code>	Return value / 返回值——无 return 则返回 <code>None</code>
<code>lambda x, y: x + y</code>	Anonymous function / 匿名函数——仅限单表达式
<code>global x</code>	Declare x as global / 声明全局变量（在函数内修改全局变量）
<code>nonlocal x</code>	Declare x from enclosing scope / 声明外层（非全局）变量

4 Data Structures / 数据结构

4.1 List / 列表—Mutable, Ordered

Syntax	Description / 说明
<code>lst = [1, 2, 3]</code>	Create list / 创建列表
<code>lst[i] / lst[-1]</code>	Index / 索引（从 0 开始；-1 最后一个）
<code>lst[i:j]</code>	Slice / 切片——左闭右开
<code>lst.append(x)</code>	Add x to end / 末尾追加
<code>lst.extend(iter)</code>	Add all from iterable / 批量追加
<code>lst.insert(i, x)</code>	Insert x at index i / 在位置 i 插入
<code>lst.remove(x)</code>	Remove first match / 删除首次出现的 x——找不到报错
<code>lst.pop(i)</code>	Remove & return element at i / 弹出位置 i 的元素（默认最后）
<code>lst.clear()</code>	Remove all elements / 清空列表
<code>lst.index(x)</code>	Find index of first match / 查找首次出现的位置
<code>lst.count(x)</code>	Count occurrences / 统计出现次数
<code>lst.sort()</code>	Sort in-place / 原地排序——返回 <code>None</code>
<code>lst.sort(reverse=True)</code>	Sort descending / 降序排序
<code>lst.sort(key=len)</code>	Sort by custom key / 按自定义规则排序
<code>sorted(lst)</code>	Return new sorted list / 返回新排序列表——不改变原列表
<code>lst.reverse()</code>	Reverse in-place / 原地反转
<code>lst.copy()</code>	Shallow copy / 浅拷贝（等价于 <code>lst[:]</code> ）
<code>del lst[i]</code>	Delete by index / 按索引删除
<code>del lst[i:j]</code>	Delete slice / 切片删除
<code>lst[i:j] = [...]</code>	Slice assignment / 切片替换

4.2 Tuple / 元组—Immutable, Ordered

Syntax	Description / 说明
<code>t = (1, 2, 3)</code>	Create tuple / 创建元组
<code>t = 1, 2, 3</code>	Parentheses optional / 圆括号可省略
<code>a, b, c = t</code>	Unpacking / 元组解包
<code>a, *rest = t</code>	Star unpacking / 星号解包—— <code>rest</code> 接收剩余元素
<code>t.count(x) / t.index(x)</code>	Count / Find index（元组仅有的两个方法）

Note: Tuples are **immutable** —no append, remove, or item assignment.

元组不可变——没有 append/remove，不能修改元素。但是，如果元组包含可变对象（如列表），该对象的内容可以修改。

4.3 Dictionary / 字典—Mutable, Key-Value Pairs

Syntax	Description / 说明
<code>d = {k: v}</code>	Create dictionary / 创建字典
<code>d = dict(zip(keys, vals))</code>	Create from two lists / 两列表 → 字典
<code>d[key]</code>	Access value / 访问值——key 不存在则 <code>KeyError</code>
<code>d.get(key, default)</code>	Safe access with fallback / 安全访问——不存在返回 <code>default</code> (默认 <code>None</code>)
<code>d[key] = val</code>	Set/update value / 设置/更新值
<code>d.keys()</code>	View of keys / 键视图 (可迭代)
<code>d.values()</code>	View of values / 值视图
<code>d.items()</code>	View of (key, value) pairs / 键值对视图
<code>for k, v in d.items(): ...</code>	Iterate / 遍历字典的标准写法
<code>d.update({k: v})</code>	Merge dict / 合并字典
<code>d.pop(key)</code>	Remove & return / 弹出键——不存在则 <code>KeyError</code>
<code>d.pop(key, default)</code>	Safe pop with fallback / 安全弹出
<code>d.popitem()</code>	Remove & return last pair (LIFO) / 弹出最后一个键值对
<code>del d[key]</code>	Delete key / 删除键
<code>key in d</code>	Check key existence / 检查键是否存在
<code>d.setdefault(key, default)</code>	Get or set default / 获取值, 不存在则设置默认值
<code>{k:v for k,v in d.items() if cond}</code>	Dict comprehension / 字典推导式

4.4 Set / 集合—Unordered, Unique Elements

Syntax	Description / 说明
<code>s = {1, 2, 3}</code>	Create set / 创建集合——空集合必须用 <code>set()</code> (<code>{}</code> 是空字典)
<code>s = set(lst)</code>	Create from iterable / 从可迭代对象创建——自动去重
<code>s.add(x)</code>	Add element / 添加元素
<code>s.remove(x)</code>	Remove element / 删除——不存在则 <code>KeyError</code>
<code>s.discard(x)</code>	Safe remove / 安全删除——不存在也不报错
<code>s.pop()</code>	Remove & return arbitrary element / 随机弹出一个元素
<code>s t</code>	Union / 并集—— <code>s.union(t)</code>
<code>s & t</code>	Intersection / 交集—— <code>s.intersection(t)</code>
<code>s - t</code>	Difference / 差集 (在 <code>s</code> 不在 <code>t</code>)
<code>s ^ t</code>	Symmetric difference / 对称差 (仅在 <code>s</code> 或仅在 <code>t</code>)
<code>s <= t</code>	Subset / 子集—— <code>s.issubset(t)</code>
<code>s >= t</code>	Superset / 超集—— <code>s.issuperset(t)</code>

4.5 Comprehensions / 推导式

Syntax	Description / 说明
	<code>pr for x in lst</code> List comprehension / 列表推导式 <code>pr for x in lst if cond</code> <code>lst if cond></code> With filter / 带过滤条件 <code>pr if cond else alt</code> <code>pr if cond else alt for x</code> <code>for x></code> With ternary / 带三元表达式（条件在左侧）
<code>{k:v for k,v in d.items()}</code>	Dict comprehension / 字典推导式
<code>{expr for x in lst}</code>	Set comprehension / 集合推导式——自动去重 <code>pr for x in lst for y in x</code> <code>重 pr for x in lst for y in x></code> Nested comprehension / 嵌套推导式——展平 二维列表
<code>any(cond) / all(cond)</code>	Any True / All True / 任一为真/全部为真

5 Modules & Packages / 模块与包

5.1 Import Variations / 导入方式

Syntax	Description / 说明
<code>import numpy</code>	Import entire module / 导入整个模块
<code>import numpy as np</code>	Import with alias / 导入并取别名（最常用）
<code>from math import sqrt, pi</code>	Import specific names / 导入特定函数/常量
<code>from math import *</code>	Import all public names / 导入所有（ 不推荐 ——污染命名空间）
<code>from math import sqrt as s</code>	Import with alias / 导入并重命名

5.2 Standard Library Highlights / 标准库核心

Syntax	Description / 说明
<code>import copy</code> <code>copy.copy(obj)</code>	Shallow copy / 浅拷贝——仅复制容器，内部元素共享
<code>import copy</code> <code>copy.deepcopy(obj)</code>	Deep copy / 深拷贝——递归复制所有内容，完全独立
<code>import math</code> <code>math.pi</code> / <code>math.e</code>	Constants / 常量：圆周率 π 和自然常数 e
<code>math.sqrt(x)</code> / <code>math.log(x, base)</code>	Square root / Logarithm / 平方根 / 对数
<code>math.ceil(x)</code> / <code>math.floor(x)</code>	Round up / Round down / 向上取整 / 向下取整
<code>math.sin(x)</code> / <code>math.cos(x)</code> / <code>math.tan(x)</code>	Trigonometric functions / 三角函数
<code>import time</code> <code>time.time()</code>	Unix timestamp / 返回自 1970-01-01 以来的秒数
<code>time.sleep(seconds)</code>	Pause execution / 暂停执行
<code>time.localtime()</code>	Current time as struct_time / 本地时间结构体
<code>import random</code> <code>random.randint(a,b)</code>	Random int between a and b (inclusive) / [a,b] 随机整数
<code>random.random()</code>	Random float in [0, 1)
<code>random.choice(lst)</code>	Pick random element / 随机选取一个元素
<code>random.shuffle(lst)</code>	Shuffle in-place / 原地打乱

5.3 pip Commands / pip 命令

Command (Terminal)	Description / 说明
<code>pip install package</code>	Install package / 安装包
<code>pip install package==1.0</code>	Install specific version / 安装指定版本
<code>pip install --upgrade pkg</code>	Upgrade package / 升级包
<code>pip uninstall package</code>	Uninstall package / 卸载包
<code>pip list</code>	List all installed packages / 列出已安装的包
<code>pip show package</code>	Show package details / 查看包详情
<code>pip freeze > requirements.txt</code>	Export dependencies / 导出依赖列表
<code>pip install -r requirements.txt</code>	Install from file / 从文件安装所有依赖

6 NumPy / 数值计算

6.1 Creating Arrays / 创建数组

Syntax	Description / 说明
<code>import numpy as np</code>	Standard import / 标准导入别名
<code>np.array([1,2,3])</code>	From list / 从列表创建
<code>np.array([[1,2],[3,4]])</code>	2D array (matrix) from nested lists / 二维数组
<code>np.arange(start, stop, step)</code>	Like range, returns ndarray / 等差数组
<code>np.linspace(s, e, num)</code>	N evenly spaced values / 等间距取 num 个点
<code>np.zeros(shape)</code>	Array of zeros / 全零数组
<code>np.ones(shape)</code>	Array of ones / 全一数组
<code>np.full(shape, value)</code>	Array filled with value / 全值数组
<code>np.eye(n)</code>	Identity matrix / $n \times n$ 单位矩阵
<code>np.random.rand(d0,d1)</code>	Uniform random [0,1) / 均匀分布随机数
<code>np.random.randn(d0,d1)</code>	Standard normal random / 标准正态分布随机数

6.2 Array Attributes & Operations / 数组属性与操作

Syntax	Description / 说明
<code>arr.shape</code>	Dimensions / 形状元组, 如 (3, 4)
<code>arr.ndim</code>	Number of dimensions / 维度数
<code>arr.dtype</code>	Data type / 元素数据类型
<code>arr.size</code>	Total number of elements / 元素总数
<code>arr.reshape(r, c)</code>	Reshape / 重塑形状——总数必须匹配, -1 自动计算
<code>arr.flatten()</code>	Flatten to 1D / 展平为一维
<code>arr.T</code>	Transpose / 转置
<code>arr.astype('float')</code>	Change dtype / 转换数据类型
<code>np.concatenate([a,b])</code>	Concatenate along axis 0 / 沿第 0 轴拼接
<code>np.stack([a,b])</code>	Stack along new axis / 沿新轴堆叠
<code>np.isclose(a, b)</code>	Element-wise close comparison / 逐元素近似相等 (容差默认 1e-8)

6.3 Vectorized Math & Aggregation / 向量化数学与聚合

Syntax	Description / 说明
<code>arr + 1 / arr * 2</code>	Broadcasting / 广播——标量自动扩展到数组每个元素
<code>arr1 + arr2</code>	Element-wise operation / 逐元素运算
<code>np.sqrt(arr) / np.log(arr)</code>	Universal functions (ufuncs) / 通用函数——逐元素计算
<code>np.sum(arr) / np.mean(arr)</code>	Sum / Mean / 总和 / 均值
<code>np.min(arr) / np.max(arr)</code>	Min / Max / 最小值 / 最大值
<code>np.std(arr) / np.var(arr)</code>	Std deviation / Variance / 标准差 / 方差
<code>np.median(arr)</code>	Median / 中位数
<code>np.sum(arr, axis=0)</code>	Sum along axis / 沿指定轴求和——0= 列方向, 1= 行方向
<code>np.percentile(arr, q)</code>	q-th percentile / 第 q 百分位数
<code>arr[arr > 5]</code>	Boolean masking / 布尔掩码过滤
<code>np.where(cond, a, b)</code>	Conditional selection / 条件选择——cond 真取 a, 假取 b

7 Object-Oriented Programming / 面向对象编程

7.1 Class Definition / 类定义

Syntax	Description / 说明
<code>class Dog:</code>	Class definition / 类定义（类名用 CamelCase）
<code>class Puppy(Dog):</code>	Inheritance / 继承——Puppy 继承 Dog 的所有属性和方法
<code>class Child(A, B):</code>	Multiple inheritance / 多重继承（从左到右 MRO 解析）

7.2 Constructor, Attributes & Methods / 构造、属性与方法

Syntax	Description / 说明
<code>def __init__(self, name):</code>	Constructor / 构造函数——创建对象时自动调用
<code>self.name = name</code>	Instance attribute / 实例属性——每个对象独立拥有
<code>cls_attr = value (in class)</code>	Class attribute / 类属性——所有实例共享
<code>def method(self, x): ...</code>	Instance method / 实例方法——第一个参数必须是 <code>self</code>
<code>obj = Dog("Buddy")</code>	Instantiation / 实例化——自动调用 <code>__init__()</code>
<code>obj.name</code>	Attribute access / 属性访问（点号表示法）
<code>obj.method(x)</code>	Method call / 方法调用——不需要传 <code>self</code>
<code>super().__init__(...)</code>	Call parent constructor / 调用父类构造函数
<code>super().method()</code>	Call parent method / 调用父类方法

7.3 Dunder (Magic) Methods / 魔术方法

Method	Description / 说明
<code>__init__(self)</code>	Constructor / 构造函数——对象创建时调用
<code>__del__(self)</code>	Destructor / 析构函数——对象销毁时调用（垃圾回收）
<code>__str__(self)</code>	String representation / 字符串表示—— <code>print(obj)</code> 和 <code>str(obj)</code> 调用
<code>__repr__(self)</code>	Developer representation / 开发者表示—— <code>repr(obj)</code> 调用
<code>__len__(self)</code>	Length / 长度—— <code>len(obj)</code> 调用
<code>__eq__(self, other)</code>	Equality / 相等比较—— <code>obj1 == obj2</code>
<code>__lt__</code> / <code>__gt__</code> / <code>__le__</code> / <code>__ge__</code>	Comparison / 比较运算符 (<, >, <=, >=)
<code>__add__</code> / <code>__sub__</code> / <code>__mul__</code>	Arithmetic / 算术运算符
<code>__getitem__(self, key)</code>	Indexing / 索引—— <code>obj[key]</code>
<code>__setitem__(self, key, v)</code>	Item assignment / 索引赋值—— <code>obj[key] = v</code>
<code>__contains__(self, item)</code>	Membership / 成员检查—— <code>item in obj</code>
<code>__call__(self)</code>	Callable / 可调用—— <code>obj()</code> 使实例像函数一样

7.4 Decorators & Encapsulation / 装饰器与封装

Syntax	Description / 说明
<code>@property</code>	Getter decorator / 将方法伪装为只读属性（不加括号访问）
<code>@xxx.setter</code>	Setter decorator / 赋值时自动调用——可在 setter 中添加验证逻辑
<code>@xxx.deleter</code>	Deleter decorator / <code>del obj.xxx</code> 时调用
<code>@staticmethod</code>	Static method / 静态方法——不需要 <code>self</code> ，不访问实例
<code>@classmethod</code>	Class method / 类方法——第一个参数是 <code>cls</code> （类本身），非 <code>self</code>
<code>_attr</code> (single underscore)	Convention: "internal use" / 约定：内部属性（不强制，靠自觉）
<code>__attr</code> (double underscore)	Name mangling / 名称改写—— <code>_ClassName__attr</code> ，减少意外访问

7.5 Polymorphism / 多态

Concept	Description / 说明
Method Overriding	Child redefines parent method / 子类重写父类方法——同名方法，不同行为
Duck Typing	"If it walks like a duck..." / 鸭子类型——不关心类型，只关心有没有需要的方法
<code>isinstance(obj, Class)</code>	Check type with inheritance support / 类型检查（支持继承判断）
<code>issubclass(A, B)</code>	Check if A is subclass of B / 检查 A 是否为 B 的子类

8 File I/O & Error Handling / 文件 IO 与 错误处理

8.1 File Operations / 文件操作

Syntax	Description / 说明
<code>open('f.txt', 'r')</code>	Read mode / 只读模式——文件必须存在
<code>open('f.txt', 'w')</code>	Write mode / 覆写模式—— 清空原有内容!
<code>open('f.txt', 'a')</code>	Append mode / 追加模式——在末尾添加
<code>open('f.txt', 'r+')</code>	Read + Write / 读写模式
<code>with open('f.txt') as f:</code>	Context Manager (Recommended) / 上下文管理器——自动关闭文件
<code>f.write(str)</code>	Write string / 写入—— 不会自动添加换行符
<code>f.writelines(list)</code>	Write list of strings / 批量写入
<code>f.read()</code>	Read entire file → string / 整个文件读为一个字符串
<code>f.read(n)</code>	Read n characters / 读取 n 个字符
<code>f.readline()</code>	Read one line (includes <code>\n</code>) / 读一行 (含换行符)
<code>f.readlines()</code>	Read all lines → list / 所有行读为字符串列表
<code>for line in f:</code>	Line-by-line iteration / 逐行遍历—— 最省内存
<code>f.seek(offset)</code>	Move file pointer / 移动文件指针
<code>f.tell()</code>	Current file pointer position / 当前指针位置
<code>f.close()</code>	Manual close / 手动关闭—— <code>with</code> 语句替代此操作
<code>f.name / f.mode / f.closed</code>	File attributes / 文件属性: 名称/模式/是否已关闭

8.2 CSV Operations / CSV 操作

Syntax	Description / 说明
<code>import csv</code>	Import CSV module / 导入 CSV 模块
<code>csv.reader(f)</code>	CSV reader → list per row / 每行返回到表
<code>csv.DictReader(f)</code>	CSV reader → dict per row / 每行返回字典—— 推荐
<code>csv.writer(f)</code>	CSV writer / CSV 写入器
<code>writer.writerow(row)</code>	Write one row / 写入一行
<code>writer.writerows(rows)</code>	Write multiple rows / 批量写入多行

8.3 Exception Handling / 异常处理

Syntax	Description / 说明
<code>try: ... except Err: ...</code>	Basic try-except / 基本异常捕获—— 必须指定异常类型
<code>except Err as e:</code>	Capture exception object / 捕获异常对象，可打印 <code>e</code>
<code>except (E1, E2):</code>	Catch multiple types / 捕获多种异常
<code>else: ...</code>	Runs only if no exception / 无异常时执行
<code>finally: ...</code>	Always runs / 无论是否异常都执行——常用于清理资源
<code>raise</code>	Re-raise current exception / 重新抛出当前异常
<code>raise ValueError("msg")</code>	Raise new exception / 抛出指定异常
<code>raise NewErr from orig</code>	Exception chaining / 异常链——保留原始异常信息

Common Exception Types / 常见异常类型:

<code>SyntaxError / IndentationError</code>	Invalid syntax / Wrong indentation (语法错误/缩进错误)
<code>NameError</code>	Variable not defined (变量未定义)
<code>TypeError</code>	Wrong type operation (类型错误——如 <code>"a"+1</code>)
<code>ValueError</code>	Valid type, invalid value (值错误——如 <code>int("abc")</code>)
<code>IndexError</code>	List index out of range (列表索引越界)
<code>KeyError</code>	Dictionary key not found (字典键不存在)
<code>AttributeError</code>	Object has no such attribute/method (属性不存在)
<code>FileNotFoundError</code>	File does not exist (文件未找到)
<code>ZeroDivisionError</code>	Division by zero (除以零)
<code>ImportError / ModuleNotFoundError</code>	Import failed (导入失败/模块未安装)
<code>StopIteration</code>	Iterator exhausted (迭代器耗尽)

9 PEP-8 Style Guide / PEP-8 编码规范

9.1 Naming Conventions / 命名约定

Element	Convention / 规范
Variables & Functions	<code>snake_case</code> —lowercase with underscores / 全小写 + 下划线
Constants	<code>UPPER_CASE_WITH_UNDERSCORES</code> / 全大写 + 下划线
Classes	<code>UpperCamelCase</code> (PascalCase) / 大驼峰 (每个单词首字母大写)
Modules & Packages	<code>short_lowercase</code> / 简短小写 (无下划线更好)
"Private" attributes	<code>_single_leading_underscore</code> / 单下划线前缀——约定为内部使用
Name mangling	<code>__double_leading_underscore</code> / 双下划线前缀——触发名称改写

9.2 Layout Rules / 布局规范

Rule	Convention / 规范
Indentation	4 spaces per level / 每级缩进 4 个空格 (不要用 Tab)
Line length	Max 79 characters / 每行最多 79 字符
Blank lines	2 between top-level functions/classes / 顶层之间 2 行空行 1 between method definitions / 方法之间 1 行空行 No more than 4 lines without blank line / 连续代码不超过 4 行
Spaces around operators	<code>x = a + b</code> —spaces around <code>=</code> , <code>+</code> , etc.
After comma	<code>f(a, b, c)</code> —space after each comma
No space in kwargs	<code>f(n=42)</code> —no spaces around <code>=</code> in keyword args
No space before (	<code>func(x)</code> —no space between function name and (
Imports	One import per line; standard library → third-party → local Group in order: standard library, third-party, local

10 Matplotlib / 数据可视化

10.1 Figure & Axes Creation / 创建画板与坐标系

Syntax	Description / 说明
<code>import matplotlib.pyplot as plt</code>	Standard import / 标准导入
<code>fig, ax = plt.subplots()</code>	Recommended style / 推荐显式写法——创建一个 Figure + 一个 Axes
<code>fig, axes = plt.subplots(r, c)</code>	Grid of subplots / 多子图网格——axes 是二维数组
<code>fig, ax = plt.subplots(figsize=(w,h))</code>	Set figure size / 设置画板尺寸 (英寸)
<code>plt.figure()</code>	Implicit figure creation / 隐式创建——不推荐

10.2 Plot Functions / 绘图函数

Syntax	Description / 说明
<code>ax.plot(x, y)</code>	Line plot / 折线图
<code>ax.scatter(x, y, s=size, c=color)</code>	Scatter plot / 散点图——s= 点大小, c= 颜色, alpha= 透明度
<code>ax.bar(x, height)</code>	Vertical bar chart / 垂直柱状图
<code>ax.barh(y, width)</code>	Horizontal bar chart / 水平柱状图
<code>ax.hist(data, bins=30)</code>	Histogram / 直方图——bins= 柱子数量
<code>ax.boxplot(data)</code>	Box plot / 箱线图——显示五数概括 + 离群点
<code>ax.pie(sizes, labels=, autopct=)</code>	Pie chart / 饼图——autopct='%1.1f%%' 显示百分比
<code>ax.fill_between(x, y1, y2)</code>	Filled area / 填充面积图
<code>ax.imshow(matrix, cmap=)</code>	Image/matrix display / 图像/矩阵显示
<code>ax.hexbin(x, y, gridsize=)</code>	Hexbin plot / 六边形分箱——大数据量散点

10.3 Styling & Decoration / 样式与装饰

Syntax	Description / 说明
<code>ax.set_title('T', fontsize=n)</code>	Set title / 设置标题
<code>ax.set_xlabel('L') / .set_ylabel('L')</code>	Set axis labels / 设置坐标轴标签
<code>ax.set_xlim(a,b) / .set_ylim(a,b)</code>	Set axis limits / 设置坐标轴范围
<code>ax.legend(loc='best')</code>	Display legend / 显示图例—— loc='best' 自动选址
<code>ax.legend(frameon=False)</code>	Legend without border / 无边框图例
<code>ax.grid(True, alpha=0.3)</code>	Add grid / 添加网格线
<code>ax.axis('off')</code>	Hide all axes / 隐藏所有坐标轴
<code>ax.spines['top'].set_visible(False)</code>	Remove top spine / 隐藏顶部边框
<code>ax.tick_params(axis='y', direction='in')</code>	Customize ticks / 自定义刻度
<code>fig.tight_layout()</code>	Auto-adjust spacing / 自动调整子图间距
<code>fig.subplots_adjust(wspace=, hspace=)</code>	Manual spacing / 手动调整间距
<code>plt.savefig('f.png', dpi=300, bbox_inches='tight')</code>	Save to file / 保存图片
<code>plt.show()</code>	Display figure / 显示图形

10.4 Colors, Markers & Linestyles / 颜色 · 标记 · 线型

Specification	Options / 选项
Color letters	'r' red, 'b' blue, 'g' green, 'c' cyan, 'm' magenta, 'y' yellow, 'k' black, 'w' white
Color hex	'#0000FF' (blue), '#FF0000' (red)
Color RGB(A)	(0, 0, 1, 0.5) —floats in [0,1], alpha = opacity
Colormaps	'viridis' (default), 'plasma', 'magma', 'jet', 'coolwarm'
Markers	'o' circle, 's' square, 'D' diamond, '^' triangle, '*' star, '.' point, ',' pixel
Linestyles	'-' solid, '--' dashed, ':' dotted, '-.' dash-dot, 'None' no line

10.5 Mathtext (LaTeX in Plots) / 数学公式

Syntax	Description / 说明
<code>r"\$</code>	
<code>sigma_i=15\$"</code>	Inline math with $\sigma_i=15$ / 行内数学公式
<code>r"\$</code>	
<code>frac{3}{5}\$"</code>	Fraction / 分数
<code>r"\$</code>	
<code>alpha\$, \$</code>	
<code>beta\$"</code>	Greek letters / 希腊字母
<code>r"\$x^{2}\$"</code>	Superscript / 上标
<code>r"\$x_{i}\$"</code>	Subscript / 下标

11 Pandas / 数据分析

11.1 Import & I/O / 导入与读写

Syntax	Description / 说明
<code>import pandas as pd</code>	Standard import / 标准导入
<code>pd.read_csv('f.csv')</code>	Read CSV → DataFrame / 读取 CSV 文件
<code>pd.read_csv('f.csv', sep=';')</code>	Read CSV with custom delimiter / 自定义分隔符
<code>pd.read_excel('f.xlsx')</code>	Read Excel → DataFrame
<code>pd.read_html(url)</code>	Scrape HTML tables / 抓取网页中的表格
<code>pd.read_json('f.json')</code>	Read JSON → DataFrame
<code>df.to_csv('f.csv', index=False)</code>	Export to CSV / 导出为 CSV (不保存行索引)

11.2 DataFrame Exploration / 数据探索

Syntax	Description / 说明
<code>df.head(n)</code> / <code>df.tail(n)</code>	First/Last n rows (default 5) / 前/后 n 行
<code>df.shape</code>	(rows, columns) / 行数和列数
<code>df.columns</code> / <code>df.index</code>	Column names / Row index / 列名列表 / 行索引
<code>df.dtypes</code>	Data type of each column / 每列的数据类型
<code>df.info()</code>	Summary: dtypes, non-null counts, memory / 详细摘要信息
<code>df.describe()</code>	Statistics: count, mean, std, min, 25%, 50%, 75%, max / 数值列统计摘要
<code>df.describe(include='object')</code>	Categorical column statistics / 分类列统计摘要
<code>df.memory_usage()</code>	Memory usage per column / 每列内存占用

11.3 Creation & Manipulation / 创建与修改

Syntax	Description / 说明
<code>pd.DataFrame(dict)</code>	From dict: keys → columns, values → data / 字典建表
<code>pd.DataFrame(list)</code>	From list: each sublist → one row / 列表建表
<code>pd.Series([v1, v2])</code>	Create Series / 创建 Series
<code>df['new'] = value</code>	Create/assign column / 创建或赋值列
<code>df.insert(i, 'name', val)</code>	Insert column at position i / 在位置 i 插入列
<code>df.drop(columns=['c'])</code>	Drop columns / 删除列—— 不修改原表 （需 <code>inplace=True</code> 或赋值）
<code>df.rename(columns={o:n})</code>	Rename columns / 重命名列
<code>df.astype('category')</code>	Change dtype / 转换列类型——低基数分类列可大幅节省内存
<code>df.assign(new=lambda x: expr)</code>	Create column via lambda / 用 lambda 创建新列（返回新表）

11.4 Selection & Indexing / 选择与索引（核心考点）

Syntax	Description / 说明
<code>df['col']</code>	Single column → Series / 单列选取
<code>df[['c1','c2']]</code>	Multiple columns → DataFrame / 多列选取
<code>df.col</code>	Attribute-style access / 属性方式访问（仅限无空格列名）
<code>df.loc[row_labels, cols]</code>	Label-based indexing / 基于标签——切片 包含 右边界
<code>df.iloc[row_pos, col_pos]</code>	Position-based indexing / 基于位置——切片 不包含 右边界
<code>df.at[label, 'col']</code>	Fast scalar access by label / 按标签取单值（最快）
<code>df.iat[i, j]</code>	Fast scalar access by position / 按位置取单值（最快）
<code>df[df['col'] > 100]</code>	Boolean filtering / 布尔过滤——选出满足条件的行
<code>df[df['col'].isin([a,b])]</code>	Filter by membership / 按值列表过滤
<code>df[df['col'].str.contains('x')]</code>	String filter / 字符串包含过滤
<code>df.query('col > 100')</code>	SQL-style query / SQL 风格查询

Critical Exam Point / 必考点： `.loc` 基于标签，切片**包含**右边界；`.iloc` 基于整数位置，切片**不包**含右边界（标准 Python 行为）。

11.5 Missing Data / 缺失值处理

Syntax	Description / 说明
<code>df.isna() / df.isnull()</code>	Boolean mask of NaN / 缺失值布尔掩码
<code>df.isna().sum()</code>	Count NaN per column / 每列缺失值计数
<code>df.dropna()</code>	Drop rows with any NaN / 删除含 NaN 的行
<code>df.dropna(subset=['c'])</code>	Drop rows where column 'c' is NaN / 指定列删 NaN
<code>df.dropna(how='all')</code>	Drop rows where ALL values are NaN / 全部为 NaN 才删
<code>df.fillna(value)</code>	Fill NaN with value / 用指定值填充
<code>df.fillna(method='ffill')</code>	Forward fill / 用前一个有效值填充
<code>df.fillna(method='bfill')</code>	Backward fill / 用后一个有效值填充
<code>df.fillna(df.mean())</code>	Fill with column mean / 用列均值填充

11.6 Aggregation & Grouping / 聚合与分组

Syntax	Description / 说明
<code>df['col'].value_counts()</code>	Frequency count / 值频次统计——常用 <code>.head(10)</code> 取 Top-N
<code>df['col'].unique()</code>	Unique values / 唯一值数组
<code>df['col'].nunique()</code>	Number of unique values / 唯一值个数
<code>df.sort_values('c')</code>	Sort by column / 按列排序—— <code>ascending=False</code> 降序
<code>df.sort_values(['c1','c2'], ascending=[True,False])</code>	Multi-column sort / 多列排序
<code>df.groupby('col').mean()</code>	Group by + aggregate / 分组聚合—— <code>mean/sum/min/max/count/std</code>
<code>df.groupby('col').agg(['mean','std'])</code>	Multiple aggregations / 多种聚合函数
<code>df.groupby(['c1','c2']).size()</code>	Group by multiple, count / 多列分组计数
<code>df.groupby('c').filter(lambda x: len(x)>10)</code>	Filter groups / 按组大小过滤
<code>df.pivot_table(index, columns, values, aggfunc)</code>	Pivot table / 透视表——多维聚合
<code>pd.crosstab(index, columns)</code>	Crosstab / 交叉表——两分类变量的 频次表

11.7 Merge, Join & Concatenate / 数据合并

Syntax	Description / 说明
<code>pd.concat([df1, df2])</code>	Concatenate along rows (axis=0) / 纵向拼接
<code>pd.concat([df1, df2], axis=1)</code>	Concatenate along columns / 横向拼接
<code>df1.join(df2, on='key')</code>	Join on index or key / 按索引或键拼接
<code>pd.merge(a, b, on='key')</code>	SQL-style merge (inner join default) / SQL 风格合并
<code>pd.merge(a, b, how='left')</code>	Left join / 左连接——保留左侧所有行
<code>pd.merge(a, b, how='right')</code>	Right join / 右连接
<code>pd.merge(a, b, how='outer')</code>	Full outer join / 全外连接
<code>pd.merge(a, b, how='inner')</code>	Inner join (default) / 内连接——仅保留匹配行
<code>pd.merge(a, b, left_on, right_on)</code>	Merge on different column names / 不同列名合并

11.8 Feature Engineering / 特征工程

Syntax	Description / 说明
<code>pd.cut(series, bins, labels)</code>	Binning / 分箱——将连续值划为离散类别
<code>pd.qcut(series, q)</code>	Quantile-based binning / 等频分箱
<code>pd.get_dummies(df, columns=['c'])</code>	One-Hot Encoding / 独热编码——每类一列
<code>LabelEncoder().fit_transform(s)</code>	Label Encoding / 标签编码（需 from sklearn.preprocessing）
<code>df['col'].astype('category').cat.codes</code>	Category codes / 分类编码（Pandas 原生）
<code>df['col'].str.extract(r'(\d+)')</code>	Regex extraction / 正则提取——从字符串中抽数字
<code>df['col'].str.replace('old', 'new')</code>	String replace / 字符串替换
<code>df['col'].str.split(',').str[0]</code>	Split and take first / 拆分取第一项

11.9 Pandas Plotting / Pandas 绘图

Syntax	Description / 说明
<code>df.plot(kind='line')</code>	Line plot (default) / 折线图——趋势/时间序列
<code>df.plot(kind='bar')</code>	Vertical bar / 垂直柱状图——分类对比
<code>df.plot(kind='barh')</code>	Horizontal bar / 水平柱状图——长标签分类
<code>df.plot(kind='hist', bins=)</code>	Histogram / 直方图——分布形态
<code>df.plot(kind='box')</code>	Box plot / 箱线图——五数概括 + 离群点
<code>df.plot(kind='scatter', x=, y=)</code>	Scatter plot / 散点图——两变量关系
<code>df.plot(kind='area')</code>	Stacked area / 堆叠面积图——构成变化
<code>df.plot(kind='pie', y=)</code>	Pie chart / 饼图——各部分占比
<code>df.plot(kind='density')</code>	KDE density plot / 核密度估计——平滑分布
<code>df.plot(kind='hexbin', x=, y=, gridsize=)</code>	Hexbin / 六边形分箱——大数据量散点
<code>df.hist(bins=, figsize=)</code>	Histogram for each column / 每列独立直方图
<code>df.boxplot(column=, by=)</code>	Box plot grouped by category / 按分类分组的箱线图
<code>pd.plotting.scatter_matrix(df)</code>	Pairwise scatter matrix / 配对散点矩阵

12 Seaborn / 统计可视化

12.1 Seaborn Basics / 基础

Syntax	Description / 说明
<code>import seaborn as sns</code>	Standard import / 标准导入
<code>sns.set_style('whitegrid')</code>	Set background style / 设置背景风格: darkgrid, whitegrid, dark, white, ticks
<code>sns.set_palette('viridis')</code>	Set color palette / 设置调色板
<code>sns.set_context('talk')</code>	Set context scale / 设置上下文缩放: paper, notebook, talk, poster
<code>sns.color_palette('Set2', n)</code>	Create custom palette / 创建自定义调色板

12.2 Seaborn Plot Functions / 绘图函数

Syntax	Description / 说明
<code>sns.barplot(data=df, x=, y=)</code>	Bar plot / 柱状图——自动计算均值 + 误差棒
<code>sns.countplot(data=df, x=)</code>	Count plot / 计数图——分类变量的频次柱状图
<code>sns.lineplot(data=df, x=, y=)</code>	Line plot / 折线图——自动聚合 + 置信区间
<code>sns.scatterplot(data=df, x=, y=, hue=)</code>	Scatter with hue / 散点图——hue 按第三变量着色
<code>sns.histplot(data=df, x=, bins=)</code>	Histogram / 直方图——比 plt.hist 更美观
<code>sns.kdeplot(data=df, x=)</code>	KDE density / 核密度估计图
<code>sns.boxplot(data=df, x=, y=)</code>	Box plot / 箱线图——hue 可进一步分组
<code>sns.violinplot(data=df, x=, y=)</code>	Violin plot / 小提琴图——箱线图 + 密度图
<code>sns.heatmap(data=matrix, annot=True)</code>	Heatmap / 热力图——annot=True 显示数值
<code>sns.pairplot(data=df)</code>	Pairwise scatter matrix / 配对散点矩阵——比 scatter_matrix 更好看
<code>sns.jointplot(data=df, x=, y=, kind='hex')</code>	Joint distribution / 联合分布图

13 Practice Set Patterns / 练习题常用模式

13.1 Data Cleaning Workflow / 数据清洗流程

Step	Code Pattern / 代码模板
1. Load data	<code>df = pd.read_csv('data.csv')</code>
2. Inspect	<code>df.head() → df.shape → df.info() → df.describe()</code>
3. Check NaNs	<code>df.isna().sum()</code>
4. Drop NaNs	<code>df = df.dropna()</code> or <code>df = df.dropna(subset=['col'])</code>
5. Rename cols	<code>df = df.rename(columns={old: new})</code>
6. Convert type	<code>df['date'] = pd.to_datetime(df['date'])</code>
7. Filter	<code>df_sub = df[df['col'] > threshold]</code>
8. Group	<code>result = df.groupby('col')['target'].sum()</code>
9. Sort	<code>result = result.sort_values(ascending=False)</code>
10. Visualize	<code>sns.barplot(data=result.head(5), x=, y=)</code>

13.2 Common Exam Patterns / 考试常见套路

Task / 任务	Pattern / 模板
Top-N ranking	<code>df['col'].value_counts().head(n)</code>
Filter + aggregate	<code>df[df['type']=='Movie']['duration'].mean()</code>
Create dict from df	<code>dict(zip(df['key'], df['value']))</code>
Dict filter	<code>{k:v for k,v in d.items() if 'text' in v}</code>
Groupby + reset	<code>df.groupby('col').sum().reset_index()</code>
Multi-condition	<code>df[(df['a']>0) & (df['b']<10)]</code> —& not and!
Date range	<code>df['date'].min(), df['date'].max()</code>
Unique values	<code>df['col'].unique(), df['col'].nunique()</code>
Regex extract	<code>df['col'].str.extract(r'(\d+)')</code>
Encode categorical	<code>pd.get_dummies(df, columns=['col'])</code>

14 Python Built-in Functions / Python 内置函数速查

14.1 Frequently Used Built-ins / 常用内置函数

Function	Description / 说明
<code>print(...)</code>	Output to console / 打印到控制台
<code>input(prompt)</code>	Read user input / 读取用户输入
<code>type(obj)</code>	Return type / 返回类型
<code>isinstance(obj, type)</code>	Type check with inheritance / 类型检查（支持继承）
<code>len(seq)</code>	Length / 长度
<code>range(start, stop, step)</code>	Integer sequence / 整数序列
<code>sum(iterable)</code>	Sum / 求和
<code>max(iterable) / min(iterable)</code>	Maximum / Minimum / 最大/最小值
<code>abs(x)</code>	Absolute value / 绝对值
<code>round(x, n)</code>	Round to n decimals / 保留 n 位小数
<code>sorted(iterable, key=, reverse=)</code>	Return new sorted list / 返回新排序列表
<code>reversed(seq)</code>	Return reverse iterator / 返回反向迭代器
<code>enumerate(seq)</code>	(index, value) pairs / 索引-值对
<code>zip(*iterables)</code>	Parallel iteration / 并行迭代
<code>map(func, iterable)</code>	Apply function to each element / 逐元素应用函数
<code>filter(func, iterable)</code>	Keep elements where func returns True / 过滤元素
<code>any(iterable) / all(iterable)</code>	Any True / All True / 任一为真 / 全部为真
<code>open(file, mode)</code>	Open file / 打开文件
<code>dir(obj)</code>	List attributes/methods / 列出属性和方法
<code>help(func)</code>	Show docstring / 显示帮助文档
<code>id(obj)</code>	Memory address / 内存地址
<code>eval(str)</code>	Evaluate string as Python code / 执行字符串中的表达式（慎用）